

Examen Módulo 2

29/04/2024

Nombre: _____ Lab: _____ Horario: _____

Este enunciado debe ser devuelto al ayudante a cargo de la sala. Presentar credencial o cédula de identidad.

Instrucciones

El presente examen tiene tres partes. Cada parte indica su puntaje y el nombre del archivo que se debe entregar. Tendrás un tiempo de 110 minutos para resolver el examen.

Debes entregar a través de Canvas un archivo para cada parte con el nombre que se especifica en el enunciado. Para ver la salida esperada en cada una de las partes del examen, puedes revisar un documento disponible en <https://bit.ly/20240429-2-ing1103>, que contiene pantallazos y algunas explicaciones.

Sistema de Puntaje

Todos los ítemes son evaluados con una escala de 1-5 (1: no presenta solución, 2: presenta esbozo de solución incompleto o con errores graves, 3: solución incompleta o imprecisa, 4: error(es) mínimo(s), 5: excelente) y luego un ponderador (1:0, 2:0,25, 3:0,5, 4:0,75, 5:1,0) es aplicado al puntaje del ítem. El puntaje otorgado a la solución se calcula como la suma de los puntajes parciales obtenidos, aplicándose los ponderadores.

Parte 1 (2 puntos)

1. [.5] Crea un programa en Python (archivo **p1-1.py**) que pida al usuario un capital C (entero mayor a cero), una tasa de rentabilidad r (valor entre 0 y 1), y una cantidad n de períodos (entero mayor a cero). Luego, en una lista llamada **simulacion**, el programa debe guardar el capital ajustado desde su valor inicial hasta el último período de acuerdo al interés compuesto. Vale decir, el primer elemento de la lista debe ser la cantidad $C_0 = C$, y el elemento C_k de la lista **simulacion** debe quedar dado por $C_k = C_0 \times (1 + r)^k$. En efecto, el segundo elemento de la lista sería $C_1 = C_0 \times (1 + r)^1$. Para el cálculo de n períodos, la lista debe contener $n + 1$ elementos, considerando que el primero es el capital inicial. Recuerda a partir de ahora revisar el documento en <https://bit.ly/20240429-2-ing1103> con la salida esperada.
2. [1.0] Ahora, extiende tu programa anterior (crea una copia y llámala **p1-2.py**), agregando la posibilidad de simular un ahorro mensual sujeto al interés compuesto. Para esto, tu programa debe pedir al usuario – además de C , r y n – un valor I a invertir recurrentemente al inicio de cada período, a partir del primero. La lista **simulacion** debe mantenerse igual que en la parte anterior, y debe agregarse al programa la lista **simulacion_ahorro**, la cual al terminar la ejecución del programa debe contener el capital actualizado con la nueva inversión en cada período y su rentabilidad. Al terminar la ejecución del programa, las dos listas deben aparecer impresas (desplegarlas directamente con **print**). Para esto, el elemento C_k de la lista **simulacion_ahorro** debe quedar en este caso dado por $C_k = (C_{k-1} + I) \times (1 + r)$, considerando $C_0 = C$. En efecto, $C_1 = (C_0 + I) \times (1 + r)$, $C_2 = (C_1 + I) \times (1 + r)$.
3. [.5] Finalmente, haz que tu programa **p1-2.py** despliegue los resultados mediante una tabla con formato en vez de listas.

Solución

Listing 1: Solución de la Parte 1-1

```
1 c = int(input("Ingrese capital: "))
2 r = float(input("Ingrese rentabilidad: "))
3 n = int(input("Ingrese períodos: "))
4
5 simulacion = [c]
6
7 i = 0
8
9 while i <= n:
10     simulacion.append(simulacion[i]*(1+r))
11     i += 1
12
13 print("-"*28)
14 print(f"C: {c}, r: {r}, n: {n}")
15 print("-"*28)
16 print(simulacion)
```

Listing 2: Solución de la Parte 1-2

```
1 c = int(input("Ingrese capital: "))
2 r = float(input("Ingrese rentabilidad: "))
3 n = int(input("Ingrese períodos: "))
4 a = int(input("Ingrese ahorro mensual: "))
5
6 simulacion = [c]
7 simulacion_ahorro = [c]
8
9 i = 0
10
11 while i <= n:
12     simulacion.append((simulacion[i]*(1+r))
13     simulacion_ahorro.append((simulacion_ahorro[i]+a)*(1+r))
14     i += 1
15
16 print("-"*28)
17 print(f"C: {c}, r: {r}, n: {n}, a: {a}")
18 print("-"*28)
19 print("Simulación:")
20 print(simulacion)
21 print("Simulación con ahorro mensual:")
22 print(simulacion_ahorro)
```

Listing 3: Solución de la Parte 1-3

```
1 c = int(input("Ingrese capital: "))
2 r = float(input("Ingrese rentabilidad: "))
3 n = int(input("Ingrese períodos: "))
4 a = int(input("Ingrese ahorro mensual: "))
5
6 simulacion = [c]
7 simulacion_ahorro = [c]
8
9 i = 0
10
11 while i <= n:
12     simulacion.append((simulacion[i]*(1+r))
13     simulacion_ahorro.append((simulacion_ahorro[i]+a)*(1+r))
14     i += 1
15
16 i = 0
17
18 print("-"*28)
19 print(f"C: {c}, r: {r}, n: {n}, a: {a}")
```

```
20 print("-"*28)
21 print("S\tSA")
22 print("-"*28)
23
24 while i <= n:
25     print(f"{round(simulacion[i], 1):5.1f}\t{round(simulacion_ahorro[i], 1)}")
26     i += 1
27
28 print("-"*28)
29 print("S: Simulación, SA: Simulación con ahorro periódico")
```

Parte 2 (p2.py; 2 puntos)

Seguiremos extendiendo el programa desarrollado en la parte anterior (p1-2.py). Para esto, crea una copia y llámala (p2.py).

1. [.5] Modifica tu programa para que en vez de pedir los datos al usuario los obtenga desde una lista llamada `parametros`. Puedes ingresar libremente los valores a esta lista para probar tu programa. Los elementos en esta lista deben ir en orden C, r, n e I .
2. [.75] El programa debe continuar simulando una inversión con ahorro periódico I . Supón que los períodos son mensuales, y que el estado cobra impuestos trimestrales (cada tercer período) a tasa t (valor decimal entre 0 y 1). Agrega a la simulación lo necesario para que se vaya actualizando la inversión en cada período considerando el impuesto trimestral. Para esto mantén todas las demás listas creadas como están y crea una nueva lista llamada `simulacion_impuesto`, que contenga los valores de todos los períodos simulados según la evolución del capital invertido bajo el régimen afecto al impuesto trimestral. El valor de la tasa t de impuesto debe mantenerse y obtenerse desde el último elemento de la lista `parametros`. Hint: Define una variable para contar que se reinicie cada tres períodos, para detectar cuándo se cobra impuesto. Para aplicar la tasa t de impuesto, debes multiplicar el capital afecto por $(1 - t)$.
3. [.5] Crea una lista llamada `resultados`, en donde el primer elemento sea la diferencia entre los valores últimos de listas `simulacion_ahorro` y `simulacion`, el segundo elemento sea la diferencia entre `simulacion_ahorro` y `simulacion_impuesto`, y el tercer elemento sea la diferencia entre `simulacion_impuesto` y `simulacion`.
4. [.25] Haz que al finalizar el programa se despliegue qué porcentaje del resultado al último período se dejó de percibir por efecto de los impuestos. Es decir, calcula dicho porcentaje basándote en la diferencia entre los últimos elementos de las listas `simulacion_ahorro` y `simulacion_impuesto`. El mensaje debe decir **Se dejó de percibir x% del resultado por impuestos**.

Recuerda revisar el documento en <https://bit.ly/20240429-2-ing1103> para ver la salida esperada del programa.

Solución

Listing 4: Solución de la Parte 2

```
1 parametros = [100, 0.1, 10, 1, 0.01]
2
3 c = parametros[0]
4 r = parametros[1]
5 n = parametros[2]
6 a = parametros[3]
```

```

7  t = parametros[4]
8
9  simulacion = [c]
10 simulacion_ahorro = [c]
11 simulacion_impuesto = [c]
12
13 i = 0
14 j = 1
15 while i <= n:
16     simulacion.append((simulacion[i])*(1+r))
17     simulacion_ahorro.append((simulacion_ahorro[i]+a)*(1+r))
18     if j == 3:
19         simulacion_impuesto.append((simulacion_impuesto[i]+a)*(1+r)*(1-t))
20         j = 1
21     else:
22         simulacion_impuesto.append((simulacion_impuesto[i]+a)*(1+r))
23         j += 1
24     i += 1
25
26 i = 0
27
28 print("-"*36)
29 print(f"C: {c}, n: {n}, a: {a}, t: {t}")
30 cabecera = f"{'S':^12}{'SA':^12}{'SI':^12}"
31 print("-"*36)
32 print(cabecera)
33 print("-"*36)
34
35 while i <= n:
36     print(f"{simulacion[i]:^12.1f}{simulacion_ahorro[i]:^12.1f}{simulacion_impuesto[i]:^12.1f}")
37     i += 1
38
39 print("\nS: Simulación, SA: Simulación con ahorro, SI: Simulación con impuesto\n")
40
41 resultados = [simulacion_ahorro[n]-simulacion[n],
42               simulacion_ahorro[n]-simulacion_impuesto[n],
43               simulacion_impuesto[n]-simulacion[n]]
44
45 cabecera = f"{'SA-S':^12}{'SA-SI':^12}{'SI-S':^12}"
46
47 print("-"*36)
48 print(cabecera)
49 print("-"*36)
50
51 print(f"{resultados[0]:^12.1f}{resultados[1]:^12.1f}{resultados[2]:^12.1f}")
52 perdida = (resultados[1] / simulacion_ahorro[n]) * 100.0
53 print(f"\nSe dejó de percibir {perdida:.1f}% del resultado por impuestos.")

```

Parte 3 (2 puntos)

Considera que las inversiones que uno realiza están afectas a un cierto nivel de riesgo por incertidumbre de las actividades económicas y del mercado, y la tasa de rentabilidad r que hemos usado es consecuencia de ese riesgo. Generalmente, inversiones con un nivel bajo de riesgo entregarán una menor rentabilidad (o generarán menores pérdidas), que inversiones más riesgosas, que pueden generar mayores rentabilidades, pero también mayores pérdidas.

- [.5] Crea una copia del programa de la parte anterior y llámala `p3-1.py`. Haz que el programa al iniciarse pregunte al usuario su “nivel de aversión al riesgo”¹, como un valor en escala de 1 (muy averso al riesgo) a 5 (nada averso al riesgo).
- [.5] Para esta parte, necesitarás usar el módulo `Math` de Python. Inclúyelo en la primera línea de tu código con:

¹Una persona más aversa al riesgo optará por realizar inversiones menos riesgosas, y vice-versa.

```
import math
```

Ahora modifica el programa para que la tasa r – para todos los cálculos (original, con ahorro recurrente I , con impuesto trimestral) sea una función del período actual, y del nivel de aversión al riesgo del usuario:

$$r(n) = A \cdot \sin(B \cdot n)$$

La función seno (`math.sin()` en Python, p.ej., `math.sin(1.52)` para obtener el seno de 1.52 radianes) utilizada en este modelo hace que la tasa r fluctúe en forma periódica, pudiendo obtener rentabilidades positivas (ganancias) o negativas (pérdidas). El factor A permite acceder a rentabilidades más altas o bajas según el nivel de riesgo. El ponderador B hace que los períodos de la función sean más cortos (mayor volatilidad y riesgo), o más largos (menor volatilidad):

$$B = \frac{2\pi}{P}$$

Usa valor de π dado por `math.pi`. Encontrarás valores de A y P en Cuadro 1). Debes considerar los valores en la siguiente tabla para la simulación de acuerdo al nivel de riesgo:

Aversión al Riesgo	A	P (Período)	Observaciones
1 (Muy Averso)	0.05	24	Menos volatilidad, períodos más largos
2	0.10	18	
3 (Neutral)	0.15	12	Ciclo completo en 12 períodos
4	0.20	6	
5 (Nada Averso)	0.25	3	Más volatilidad, períodos más cortos

Cuadro 1: Valores ajustados según la aversión al riesgo

Puedes guardar la información anterior en listas (p.ej., una lista para A y otra para P), a fin de facilitar la programación de la simulación.

- [1.0] Crea una copia final del programa, de nombre `p3-2.py`. Elimina el mensaje final que habías agregado antes sobre el porcentaje de la inversión que se dejó de percibir por impuestos. Además, puedes eliminar los bloques que calculan dicho porcentaje, junto con la lista `resultados`. Mantén todas las otras listas creadas (`simulacion`, `simulacion_ahorro`, `simulacion_impuesto`, etc.) funcionando como en las partes anteriores.

Modifica el programa para que deje de preguntar al usuario su nivel de aversión al riesgo, y en cambio, para cada nivel de riesgo (1 a 5) calcule el resultado final de la inversión original, con ahorro recurrente y con impuesto trimestral, y vaya guardando para cada nivel de riesgo estos resultados en listas `resultado_original`, `resultado_ahorro`, `resultado_impuesto`. Es decir, cada una de estas listas debe contener cinco valores al terminar la ejecución, y los valores van calculados en orden decreciente de aversión al riesgo.

Recuerda revisar el documento en <https://bit.ly/20240429-2-ing1103> para ver la salida esperada del programa.

Solución

Listing 5: Solución de la Parte 3-1

```
1 import math
2
3 parametros = [100, 0.05, 10, 1, 0.01]
4 lst_P = [24, 18, 12, 6, 3]
```

```

5
6 c = parametros[0]
7 r = parametros[1]
8 n = parametros[2]
9 a = parametros[3]
10 t = parametros[4]
11
12 aversion = int(input("Ingresa tu nivel de aversión al riesgo: "))
13
14 simulacion = [c]
15 simulacion_ahorro = [c]
16 simulacion_impuesto = [c]
17
18 i = 0
19 j = 1
20
21 b = (2 * math.pi) / (lst_P[aversion-1])
22
23 while i <= n:
24     r = (aversion * 0.05) * math.sin(b*i)
25     simulacion.append((simulacion[i])*(1+r))
26     simulacion_ahorro.append((simulacion_ahorro[i]+a)*(1+r))
27     if j == 3:
28         simulacion_impuesto.append((simulacion_impuesto[i]+a)*(1+r)*(1-t))
29         j = 1
30     else:
31         simulacion_impuesto.append((simulacion_impuesto[i]+a)*(1+r))
32         j += 1
33     i += 1
34
35 i = 0
36
37 print("-"*36)
38 print(f"C: {c}, n: {n}, a: {a}, t: {t}, ar: {aversion}")
39 cabecera = f"{'S':^12}{'SA':^12}{'SI':^12}"
40 print("-"*36)
41 print(cabecera)
42 print("-"*36)
43
44 while i <= n:
45     print(f"{simulacion[i]:^12.1f}{simulacion_ahorro[i]:^12.1f}{simulacion_impuesto[i]:^12.1f}")
46     i += 1
47
48 print("\nS: Simulación, SA: Simulación con ahorro, SI: Simulación con impuesto\n")
49
50 resultados = [simulacion_ahorro[n]-simulacion[n],
51               simulacion_ahorro[n]-simulacion_impuesto[n],
52               simulacion_impuesto[n]-simulacion[n]]
53
54 cabecera = f"{'SA-S':^12}{'SA-SI':^12}{'SI-S':^12}"
55
56 print("-"*36)
57 print(cabecera)
58 print("-"*36)
59
60 print(f"{resultados[0]:^12.1f}{resultados[1]:^12.1f}{resultados[2]:^12.1f}")
61 perdida = (resultados[1] / simulacion_ahorro[n]) * 100.0
62 print(f"\nSe dejó de percibir {perdida:.1f}% del resultado por impuestos.")

```

Listing 6: Solución de la Parte 3-2

```

1 import math
2
3 parametros = [100, 0.05, 24, 1, 0.01]
4 lst_P = [24, 18, 12, 6, 3]
5
6 c = parametros[0]
7 r = parametros[1]
8 n = parametros[2]
9 a = parametros[3]
10 t = parametros[4]

```

```
11
12 resultado = []
13 resultado_ahorro = []
14 resultado_impuesto = []
15
16 aversion = 1
17
18 while aversion <= 5:
19     simulacion = [c]
20     simulacion_ahorro = [c]
21     simulacion_impuesto = [c]
22
23     b = (2 * math.pi) / (lst_P[aversion-1])
24     i = 0
25     j = 1
26     while i <= n:
27         r = (aversion * 0.05) * math.sin(b*i)
28         simulacion.append((simulacion[i])*(1+r))
29         simulacion_ahorro.append((simulacion_ahorro[i]+a)*(1+r))
30         if j == 3:
31             simulacion_impuesto.append((simulacion_impuesto[i]+a)*(1+r)*(1-t))
32             j = 1
33         else:
34             simulacion_impuesto.append((simulacion_impuesto[i]+a)*(1+r))
35             j += 1
36         i += 1
37     aversion += 1
38     resultado.append(simulacion[n])
39     resultado_ahorro.append(simulacion_ahorro[n])
40     resultado_impuesto.append(simulacion_impuesto[n])
41
42 i = 1
43
44 print("-"*50)
45 print(f"C: {c}, n: {n}, a: {a}, t: {t}")
46 print("-"*50)
47 cabecera = f"{'AR':^5}{ 'RES':^10}{ 'RES_AHORRO':^15}{ 'RES_IMPUESTO':^15}"
48 print(cabecera)
49 print("-"*50)
50
51 while i <= 5:
52     print(f"{'i':^5}{resultado[i-1]:^10.1f}{resultado_ahorro[i-1]:^15.1f}{resultado_impuesto[i-1]:^15.1f}↵")
53     i += 1
54
55 print("\nAR: Aversión al riesgo, RES: Resultado\n")
```